CLAUDE CERTIFIED ARCHITECT – FOUNDATIONS

Comprehensive Study Guide

CCAR-F | Exam blueprint, architecture patterns, practical examples, decision frameworks and practice questions

Source basis Structured around the user-provided Claude Certified Architect – Foundations Exam Guide, Version 1.0, effective July 2026. Explanations and examples are instructional elaborations; exam objectives and stated exam facts are taken from that guide.

Important Anthropic product behavior, CLI flags, SDK APIs and configuration syntax can change. Treat the supplied exam guide as the study blueprint and verify implementation syntax against current Anthropic documentation before production use.

How to use this guide

- Study Domain 1 first: it carries the highest exam weight (27%) and establishes the agent architecture used in several scenarios.
- Study each concept using the sequence: definition → architecture decision → failure mode → implementation pattern → exam trap.
- Do the practical labs. The exam is scenario-based; memorizing terms without choosing the right tradeoff is insufficient.
- Complete the practice questions without looking at the answer key. For every wrong answer, explain why the distractors are wrong.
- Use the final revision sheets during the last 48 hours.

Exam blueprint and logistics

Exam attribute	Guide specification
Credential / code	Claude Certified Architect – Foundations / CCAR-F
Items and format	60 multiple-choice and multiple-response items; each item states how many responses to select
Structure	4 scenarios selected from a bank of 6
Time	120 minutes
Passing standard	Scaled score 720 on a 100–1,000 scale
Validity	12 months from award
Domains	Agentic Architecture 27%; Tool Design & MCP 18%; Claude Code 20%; Prompt Engineering 20%; Context & Reliability 15%

Exam strategy: allocate time by complexity, not equally by question. Read the scenario constraints first (latency, risk, policy, context size, tool availability), then identify the root cause. Prefer the smallest architectural change that directly addresses that root cause. Avoid adding infrastructure when a better tool description, explicit criteria, or deterministic prerequisite solves the problem.

Architecture mental model

Think of a production Claude application as five cooperating layers:

Layer	Responsibility	Typical components
Model interaction	Reasoning, response generation, tool-use decisions	Claude API, system prompt, messages, stop_reason
Agent runtime	Owns the loop, delegates work, manages sessions	Claude Agent SDK, coordinator, subagents, hooks
Tool integration	Provides controlled access to systems and data	MCP tools/resources, built-in tools, APIs
Policy and workflow control	Enforces prerequisites, authorization and escalation	Hooks, gates, validators, human approval
Reliability and operations	Controls context, retries, observability and review	Structured errors, scratchpads, provenance, sampling

Core architecture principle Use the model for flexible reasoning and selection. Use ordinary program logic for rules that must be enforced deterministically. Do not ask a prompt to guarantee a business invariant.

Domain 1 — Agentic Architecture & Orchestration (27%)

1.1 Agentic loop lifecycle

An agentic loop is the runtime cycle that lets Claude decide whether it needs a tool, receive the tool result, and continue reasoning. The application—not the model—executes the requested tool and sends the result back.

```
messages = initial_messages
while True:
  response = call_claude(messages, tools=tool_definitions)
  messages.append(response)

  if response.stop_reason == "end_turn":
    return extract_final_answer(response)

  if response.stop_reason == "tool_use":
    for tool_call in response.tool_calls:
      result = execute_tool(tool_call.name, tool_call.input)
      messages.append(tool_result(
        tool_use_id=tool_call.id,
        content=result
      ))
    continue

  handle_other_stop_reason(response.stop_reason)
```

Signal	Meaning	Runtime action
stop_reason = tool_use	Claude has requested one or more tools	Execute the requested tool(s), append corresponding tool results, call Claude again
stop_reason = end_turn	Claude has completed its turn	Return the final response; stop the loop
Other stop reason	A different termination condition occurred	Handle explicitly according to API/runtime semantics; do not assume completion

- Tool results must be linked to the corresponding tool-use request and appended to the conversation history.
- The next model call must include the conversation history needed for coherent reasoning.
- The model decides which tool to call based on context; this is different from a hard-coded sequence or decision tree.
- Do not parse phrases such as 'I'm done' to decide whether to terminate.
- Do not use assistant text content as the completion signal.
- An arbitrary iteration cap is not the primary termination mechanism. A defensive safety limit may exist, but it must not replace correct stop-reason handling.

Exam pattern If the scenario says the agent stops after requesting a tool, inspect the loop. The likely defect is failure to execute the tool, append its result, and continue while stop_reason is tool_use.

1.2 Coordinator–subagent architecture

Use a hub-and-spoke design for multi-agent workflows. The coordinator decomposes the request, decides which specialists are needed, invokes them, handles errors, routes information and synthesizes the final result. Subagents do not automatically inherit the coordinator's conversation history.

Component	Owns	Should not do
Coordinator	Scope, task decomposition, delegation, context handoff, recovery, aggregation, final quality check	Delegate blindly to every agent on every request
Research/search subagent	Find sources and return claims plus source metadata	Write the final report without synthesis requirements
Document-analysis subagent	Extract evidence from supplied documents	Assume it can see documents or history not passed to it
Synthesis subagent	Combine supplied findings, preserve attribution, identify gaps	Invent missing evidence or silently reconcile conflicts

Good coordinator behavior:

- Dynamically select subagents based on query complexity and required coverage.
- Decompose broad topics into a complete scope map before delegation. Avoid assigning only a narrow subset of a broad topic.
- Assign distinct subtopics or source types to reduce duplicated work.
- Pass the complete relevant findings and metadata explicitly in each subagent prompt.
- Launch independent subagents in parallel when their tasks do not depend on one another.
- Evaluate synthesis for missing coverage; issue targeted follow-up tasks where gaps remain.
- Route inter-agent communication through the coordinator for observability and consistent error handling.

```
Coordinator task handoff (illustrative structure):
{
 "objective": "Assess the market and produce a cited report",
 "scope": ["North America", "Europe", "Asia-Pacific"],
 "agent_assignment": "Research the Europe segment only",
 "constraints": ["Use sources published within the last 24 months"],
 "output_contract": {
  "claims": [{"claim": "...", "evidence": "...",
          "source_url": "...", "published_at": "..."}],
  "coverage_gaps": []
 }
}
```

Critical detail A subagent has isolated context. Put the necessary facts, prior findings, constraints and source metadata directly into its task prompt. Do not rely on implicit parent memory.

1.3 Spawning, tools and context passing

- The Task tool is the mechanism identified in the exam guide for spawning subagents.
- The coordinator must have Task included in allowedTools to invoke subagents.
- AgentDefinition describes each specialist: description, system prompt and tool restrictions.
- Multiple independent Task calls can be emitted in one coordinator response to run in parallel.
- Use structured outputs to separate content from metadata such as URLs, filenames, page numbers, timestamps and methodology.

- Fork-based sessions let you explore divergent approaches from a shared baseline without conflating their histories.

Bad handoff: 'Continue the research from the parent conversation.'

Good handoff: include the research question, scope, all relevant prior findings, evidence references, constraints, and the required output schema.

1.4 Workflow enforcement and human handoff

Prompt-based guidance is probabilistic. A prerequisite that protects money, identity, access or safety needs programmatic enforcement.

Requirement	Recommended control
Verify identity before looking up private orders	Gate lookup_order until get_customer returns a verified customer ID
Block refund above policy threshold	Intercept the tool call in code/hook and deny or route to human review
Require approval before irreversible action	Explicit approval gate outside model discretion
Escalate a policy exception	Structured human handoff with case facts and recommended next action

A useful handoff packet contains customer/case ID, verified identity status, issue summary, relevant transaction identifiers, amounts and dates, root cause, actions already attempted, tool results, policy gap, recommended action and urgency. A human should not need to reconstruct the case from a long transcript.

```
Example policy gate:
if tool.name == "process_refund":
  require_verified_customer_id()
  if refund_amount > configured_limit:
    block_tool_call()
    create_human_escalation(reason="Refund exceeds authority")
  else:
    allow_tool_call()
```

1.5 Agent SDK hooks

Hook pattern	Use	Why
Pre-tool / tool-call interception	Inspect or block an outgoing tool call before execution	Enforce authorization, prerequisites, thresholds and policy
PostToolUse	Transform a tool result before the model consumes it	Normalize timestamps, status codes, inconsistent field formats
Other lifecycle hooks	Observe or influence defined runtime lifecycle points	Use only where the SDK exposes the relevant lifecycle event

Use hooks when the requirement is deterministic. Example: normalize Unix timestamps to ISO 8601, map numeric status codes to canonical labels, or block a refund above $500. Prompt instructions can explain policy but cannot provide a hard guarantee.

Exam trap If the scenario says 'must never', 'guarantee', 'block', 'before any financial operation', or 'policy threshold', choose programmatic enforcement rather than a stronger prompt.

1.6 Task decomposition

Pattern	Use when	Example
Prompt chaining / fixed sequential pipeline	Steps are predictable and order is known	Analyze each source file, then perform a cross-file integration review
Dynamic adaptive decomposition	The next task depends on discoveries made along the way	Explore a legacy system, map dependencies, then prioritize test gaps
Parallel decomposition	Tasks are independent and can be performed simultaneously	Research separate geographic markets or analyze separate documents
Coordinator refinement loop	Initial findings may be incomplete	Synthesize, detect missing coverage, delegate targeted research, synthesize again

For large code reviews, use per-file local analysis followed by a separate cross-file pass. This limits attention dilution while retaining a view of integration and data-flow problems.

1.7 Session state, resumption and forking

Approach	Use when	Risk / control
Resume named session (--resume <session-name>)	Prior conversation and tool findings remain valid	Tell Claude which files changed since the session
Fork session	Explore competing designs from the same baseline	Keep branches independent and compare outputs
New session + structured summary	Prior tool results are stale or context is degraded	Inject current facts and findings rather than trusting old observations

A resumed session can contain stale facts. If files changed, explicitly identify the changed paths and request targeted re-analysis. For long-running work, persist findings in scratchpad/state files and load a manifest on recovery.

Domain 2 — Tool Design & MCP Integration (18%)

2.1 Design tool interfaces for reliable selection

Tool descriptions are a primary signal Claude uses to decide which tool to call. A name and one-line description are insufficient when tools overlap.

Description element	What to specify
Purpose	Exact job the tool performs
Inputs	Accepted identifiers, formats, required and optional fields
Examples	Representative natural-language requests and valid input examples
Boundaries	What the tool does not do; when a similar tool is preferred
Output	Returned fields, units, status meanings and empty-result behavior
Side effects	Whether the operation reads, writes, charges, refunds, deletes or triggers external work

Example of differentiated descriptions:

```
get_customer:
"Resolve and verify a customer identity using an exact customer ID
or verified email. Returns customer_id and verification_status.
Use before accessing account-specific orders. Do not use this tool
to retrieve an order by order number."

lookup_order:
"Retrieve order status and line items using an exact order number.
Use for requests such as 'where is order 12345?' This tool does not
verify customer identity and must not be used to authorize refunds."
```

- If descriptions are ambiguous, improve them before adding a routing classifier.
- Split a generic tool into purpose-specific tools when their contracts or side effects differ.
- Rename overlapping tools to make their scope obvious.
- Review system prompt wording for keyword-sensitive instructions that accidentally bias tool selection.
- Do not combine unrelated operations merely to reduce tool count if it weakens boundaries or authorization.

2.2 MCP errors and recovery

MCP tool failures should be structured. A generic 'operation failed' gives the agent no basis for deciding whether to retry, ask for corrected input, explain a policy denial or escalate.

Error category	Example	Retry?	Agent behavior
Transient	Timeout, service unavailable, temporary network failure	Usually yes, bounded retry/backoff	Retry or use an alternative; preserve partial results
Validation	Malformed order ID, missing required input	No, until input is corrected	Ask for corrected input or explain required format
Business	Refund exceeds policy, order is non-refundable	No	Explain policy or escalate if exception requested
Permission	Caller lacks required role or access	Not blindly	Do not retry unchanged; request authorization or escalate
Empty result	Valid query but no matching records	Not an error	Report no match; request another identifier if needed

```
Illustrative MCP error payload:
{
 "isError": true,
 "errorCategory": "transient",
 "isRetryable": true,
 "message": "Order service timed out",
 "attemptedOperation": "lookup_order",
 "partialResults": null,
 "suggestedAlternative": "Retry with the same order ID"
}
```

The guide specifically calls out the MCP isError flag and structured metadata such as errorCategory, isRetryable and a human-readable description. Keep the distinction between a failed query and a successful query that returned zero matches.

Subagents should recover locally from transient failures where appropriate. Escalate only unresolved errors, with what was attempted and any partial results.

2.3 Tool distribution and tool_choice

Setting / pattern	Meaning	Use
tool_choice: auto	Claude may choose a tool or respond with text	Normal flexible agent interaction
tool_choice: any	Claude must call a tool, but can choose which available tool	Guarantee structured tool use when the correct schema is one of several options
Forced tool selection	Require a named tool	Ensure a specific first operation, such as metadata extraction
Scoped tool access	Give each agent only role-relevant tools	Reduce selection complexity and prevent specialist misuse

The guide warns that too many tools increase decision complexity. A specialist often needs four or five relevant tools, not every tool in the system. Give a synthesis agent a narrow verify_fact tool for simple checks if needed; route complex research through the coordinator.

Least privilege Tool availability is an architectural boundary. Restricting tools reduces accidental misuse and limits the impact of a mistaken model decision.

2.4 MCP servers, tools and resources

MCP is the integration protocol used to expose tools and resources to Claude-enabled applications. In this exam, distinguish the two:

MCP concept	Purpose	Example
Tool	An operation the agent can invoke	Create ticket, retrieve order, run query, process refund
Resource	A content object/catalog the client can read	Issue summaries, documentation hierarchy, database schema catalogue
Server	A provider that exposes tools/resources	Jira integration server or internal operations server

Configuration scope	Location in exam guide	Typical purpose
Project-level	.mcp.json	Shared team integration configuration committed with the repository
User-level	~/.claude.json	Personal or experimental server configuration

- Use environment-variable expansion such as ${GITHUB_TOKEN}; do not commit secrets.
- Configured servers' tools are discovered at connection time and can be available simultaneously.
- Expose catalogs and hierarchies as resources to reduce exploratory calls.
- Prefer an established community MCP server for standard integrations; build custom servers for team-specific workflows.
- Write descriptions that explain the MCP tool's capabilities and outputs so the agent does not default to a less capable built-in tool.

Illustrative project configuration (shape only; validate exact current syntax against the current Claude Code documentation):

```
{
 "mcpServers": {
  "work-tracker": {
   "command": "npx",
   "args": ["-y", "example-mcp-server"],
   "env": {
    "API_TOKEN": "${WORK_TRACKER_TOKEN}"
   }
  }
 }
}
```

2.5 Built-in tools: choose the right one

Tool	Best use	Example
Glob	Find paths matching file patterns	Find all **/*.test.tsx files
Grep	Search file contents for text/patterns	Find every call to ProcessRefund or a specific error string
Read	Load file contents	Inspect a controller, service or test file
Edit	Targeted replacement anchored on unique text	Change one known method or configuration block
Write	Create or replace file contents	Create a new file or rewrite after a safe read
Bash	Execute shell commands	Run tests, inspect git diff, build or execute approved commands

For codebase exploration, start with Glob/Grep to find likely entry points, then Read to trace imports, callers and dependencies. Do not read every file upfront. If Edit fails because the target text is not unique, Read the file and use a controlled Write or a more specific unique anchor.

Domain 3 — Claude Code Configuration & Workflows (20%)

3.1 CLAUDE.md hierarchy and organization

Scope	Location in guide	Who it affects
User	~/.claude/CLAUDE.md	Only the current developer; not shared by version control
Project	Root CLAUDE.md or .claude/CLAUDE.md	Team/project instructions when committed
Directory	CLAUDE.md in a subdirectory	Instructions relevant to that directory tree

Use project-level instructions for shared standards: architecture conventions, build/test commands, code review expectations, fixture usage and repository-specific practices. Use user-level instructions for personal preferences. Use directory-level instructions for local ownership and conventions.

- Use @import to keep instruction files modular and include standards relevant to a package.

- Use .claude/rules/ for focused topic-specific rule files such as testing.md, api-conventions.md and deployment.md.
- Use /memory to inspect which memory files are loaded when behavior differs across sessions.
- Avoid a huge monolithic instruction file that mixes every package's conventions.

3.2 Commands and skills

Mechanism	Location	When to use
Project slash command	.claude/commands/	Reusable team workflow committed to source control
Personal slash command	~/.claude/commands/	Developer-specific workflow
Project skill	.claude/skills/<skill>/SKILL.md	On-demand, reusable task procedure for the project
Personal skill	~/.claude/skills/	Personal variation; use a distinct name to avoid changing team behavior
CLAUDE.md	Project/user/directory locations	Always-loaded or scope-loaded instructions and standards

The guide identifies skill frontmatter fields:

Frontmatter	Purpose
context: fork	Run the skill in an isolated subagent context; useful for verbose exploration or brainstorming
allowed-tools	Restrict tools available during skill execution
argument-hint	Tell the user which parameters to provide

Choose CLAUDE.md for universal instructions that should be present consistently. Choose a skill for a specific workflow that is invoked when needed. Use context: fork when the task's output is lengthy or exploratory and should not pollute the main session.

3.3 Path-specific rules

Rules in .claude/rules/ can use YAML frontmatter with glob path patterns. They load conditionally when the agent works on matching files. This is more maintainable than directory-only instructions when the same convention applies across files scattered throughout the repository.

```
---
paths:
  - "src/api/**/*"
---
API conventions:
- Use the established async request pattern.
- Return the standard error contract.

---
paths:
  - "**/*.test.tsx"
---
Test conventions:
- Prefer user-visible behavior assertions.
```

```
- Reuse shared fixtures where available.
```

Use path-scoped rules for cross-cutting file types (for example all tests, Terraform files or API handlers) regardless of directory. Use a subdirectory CLAUDE.md when conventions are genuinely tied to that directory.

3.4 Plan mode versus direct execution

Plan mode	Direct execution
Large multi-file changes	Small, clear, well-scoped change
Architecture or design decisions	Known implementation path
Migration, refactor, multiple valid approaches	Single validation check or isolated bug fix
Explore dependencies before editing	Implement immediately when scope and impact are understood
Use Explore subagent for verbose discovery	Avoid unnecessary planning overhead

For a .NET monolith-to-microservices restructuring, a framework migration across dozens of projects, or a Terraform architecture change, use plan mode first. Explore the repository, map dependencies, identify risks and propose a sequence. Then execute the agreed plan. For a one-line validation fix with a known stack trace, direct execution is appropriate.

3.5 Iterative refinement

- Provide 2–3 concrete input/output examples when prose descriptions are interpreted inconsistently.
- Write tests for expected behavior, edge cases and performance requirements before implementation; feed failures back into the next iteration.
- Use the interview pattern: ask Claude to identify unanswered design questions and risks before implementation.
- Address interacting defects together; fix independent defects sequentially to simplify diagnosis.
- For edge cases, provide exact input and expected output rather than saying 'handle nulls correctly'.

Example: instead of 'normalize customer names', provide inputs such as ' smith, jane ' and expected output 'Jane Smith', and explicitly define handling of null, empty, initials, accents and multi-part surnames.

3.6 Claude Code in CI/CD

The exam guide emphasizes non-interactive execution and machine-readable output.

```
claude -p "Review this pull request for security and correctness"

# Structured output for pipeline parsing:
claude -p "Review the changed files"   --output-format json   --json-schema ./review-schema.json
```

- Use -p / --print in CI to avoid waiting for interactive input.
- Use --output-format json and --json-schema for machine-parseable findings.
- Put repository testing standards, fixtures and review criteria in CLAUDE.md.
- Use an independent review session rather than asking the same context that generated code to review itself.
- When re-running after new commits, pass previous findings and ask for only new or still-unaddressed issues.
- Provide existing tests so generated tests do not duplicate coverage already present.

Example CI review schema fields: file_path, line_start, line_end, category, severity, finding, evidence, suggested_fix, detected_pattern. The schema must be validated by the consumer; syntactically valid JSON does not guarantee a true finding.

Domain 4 — Prompt Engineering & Structured Output (20%)

4.1 Explicit criteria and precision

Vague instructions such as 'be conservative', 'review carefully' or 'only report high-confidence issues' do not define a decision boundary. Explicit criteria do.

Weak instruction	Stronger instruction
Check comments for accuracy	Report a comment only when its claimed behavior contradicts the implementation; skip stylistic wording differences
Find serious bugs	Report defects that can cause incorrect persisted state, data loss, authorization bypass or externally visible incorrect behavior
Be conservative	Do not report formatting, naming preferences or patterns consistent with established local conventions
Use high severity appropriately	Critical = exploitable security issue or irreversible data loss; high = major functional failure with broad impact; medium = limited functional defect

False positives reduce trust in the entire review system. If a category is persistently noisy, temporarily disable it while improving its criteria rather than flooding developers with low-value findings.

4.2 Few-shot prompting

Few-shot examples demonstrate the desired judgment and output format. They are especially useful for ambiguous cases, inconsistent document layouts, and distinguishing acceptable patterns from actual defects.

- Use 2–4 targeted examples for ambiguous decisions.
- Show why an action was chosen over plausible alternatives.
- Include both positive and negative examples: what to flag and what to skip.
- Demonstrate the exact output fields, severity labels and evidence format.
- Vary document structures so extraction generalizes beyond one template.

```
Example review instruction:
Report only defects that meet one of these criteria:
1. Incorrect business behavior compared with the stated requirement.
2. Authorization or data-isolation failure.
3. Resource leak or unbounded resource consumption.
4. Breaking API contract change without compatibility handling.

Do not report formatting, naming preferences, or a pattern already
used consistently in the surrounding module.

For every finding return: file, line, category, severity,
evidence, impact, suggested fix.
```

4.3 Structured output with tool_use and JSON Schema

For reliable schema-constrained output, define a tool with JSON Schema parameters and use tool_use. The guide treats this as more reliable than asking Claude to print JSON in ordinary text.

tool_choice	Guarantee
auto	Claude may respond with text rather than call a tool
any	Claude must call a tool, but can choose among available tools

Forced named tool	Claude must call the specific named tool

Strict schemas eliminate JSON syntax errors, not semantic errors. A payload can conform to the schema while containing a wrong amount, an invalid date relationship or line items that do not sum to the total.

Schema design principles:

- Make truly mandatory fields required.
- Use optional or nullable fields when source information may be absent; this reduces pressure to fabricate.
- Use enums for stable categories. Add 'other' plus a detail field when categories may evolve.
- Add 'unclear' where the source is ambiguous.
- Specify normalization rules in the prompt (currency, dates, units, whitespace, identifiers).
- Validate relationships and business invariants in code after schema validation.

Illustrative extraction schema:

```
{
 "type": "object",
 "properties": {
  "invoice_number": {"type": ["string", "null"]},
  "invoice_date": {"type": ["string", "null"]},
  "currency": {"type": "string", "enum": ["USD", "EUR", "GBP", "other"]},
  "currency_other_detail": {"type": ["string", "null"]},
  "stated_total": {"type": ["number", "null"]},
  "line_items": {
   "type": "array",
   "items": {
    "type": "object",
    "properties": {
     "description": {"type": "string"},
     "amount": {"type": "number"}
    },
    "required": ["description", "amount"]
   }
  }
 },
 "required": ["invoice_number", "invoice_date", "currency",
         "currency_other_detail", "stated_total", "line_items"]
}
```

Note: JSON Schema dialects and strict-mode requirements can differ by API version. The example illustrates the design pattern; check the current API schema constraints before copying into production.

4.4 Validation, retry and feedback loops

Use a validation pipeline with distinct stages:

- Schema validation: required fields, types, enum membership and structural constraints.
- Semantic validation: totals reconcile, dates are plausible, identifiers match expected formats, and extracted values are assigned to the correct fields.
- Source validation: every extracted fact is supported by the source and not inferred without authorization.
- Business validation: downstream rules and thresholds are satisfied.

On a resolvable failure, retry with the original document, failed extraction and exact validation errors. Retry can correct formatting or structural mistakes. It cannot recover information absent from the supplied source; retrieve the missing source or return null/unclear.

```
Validation retry loop:
1. Extract structured fields.
2. Validate schema and semantic invariants.
3. If fixable:
  - send original source
  - send prior extraction
  - send precise validation errors
  - request corrected extraction
4. If source evidence is absent:
  - do not retry repeatedly
  - return null / unclear or route to human review
5. Record error category and correction outcome.
```

For automated code review feedback, include detected_pattern so dismissed findings can be grouped and analyzed. For extraction, return stated_total and calculated_total plus conflict_detected to make inconsistencies explicit.

4.5 Message Batches API

Characteristic in supplied guide	Implication
50% cost savings	Useful for high-volume, non-blocking workloads
Up to 24-hour processing window	Do not use for workflows that require an immediate result
No guaranteed latency SLA	Cannot be relied on for a blocking pre-merge gate
No multi-turn tool calling within one request	Not appropriate where the model must call tools, receive results and continue in the same batch request
custom_id	Correlates each response with its submitted request

Workload	Recommended API pattern
Pre-merge security/code check	Synchronous API / immediate response
Nightly technical debt report	Message Batches API
Weekly document classification	Message Batches API
Customer waiting for refund decision	Synchronous agent workflow with policy gates
Overnight test generation	Message Batches API if no blocking SLA

Before a large batch, tune prompts on a representative sample. For failures, resubmit only failed custom_id items with appropriate changes, such as chunking oversized documents. Calculate submission cadence against the full SLA, including the maximum processing window and downstream review time.

4.6 Independent and multi-pass review

- A model that generated code retains its generation context; self-review may be biased toward its prior choices.
- Use a second independent Claude instance without the generator's reasoning context for a stronger review.
- For large changes, perform focused per-file reviews for local defects and a separate integration pass for cross-file data flow.
- Confidence scores can help route findings, but confidence must be calibrated and should not replace evidence or deterministic validation.
- Do not require multiple independent runs to agree on every bug; consensus filters can suppress intermittent but real defects.

Domain 5 — Context Management & Reliability (15%)

5.1 Preserve critical facts across long interactions

Long conversations accumulate tool output and can degrade. Summaries are useful but can lose exact dates, amounts, identifiers and customer commitments. Preserve critical facts separately from narrative summaries.

Context layer	What belongs there
Case facts	Exact IDs, amounts, dates, statuses, customer commitments, verification state
Conversation summary	Goals, decisions, unresolved questions and interaction history
Tool evidence	Only fields needed for the next decision, with source metadata
Scratchpad / state	Exploration findings, dependency maps, current phase, completed tasks and next actions
Provenance	Claim-to-source mapping, publication date, page or URL, evidence excerpt

- Trim verbose tool output before adding it to the conversation. Keep only relevant fields.
- Put key findings near the beginning of aggregated inputs and use explicit section headings.
- Preserve exact transactional facts in a persistent case-facts block included in each prompt.
- Pass structured facts, citations and relevance scores downstream rather than verbose reasoning.
- Include full conversation history in subsequent API requests when needed for conversational coherence.

Lost in the middle The guide highlights that information near the beginning and end of long inputs is often processed more reliably than information buried in the middle. Put key findings first, structure the rest with headings, and avoid unnecessary tool output.

5.2 Escalation and ambiguity resolution

Situation	Correct behavior
Customer explicitly asks for a human	Honor the request immediately; do not force another investigation first
Straightforward issue within policy	Offer/perform autonomous resolution even if customer is frustrated, unless they reiterate human preference
Policy exception or policy silence	Escalate; do not invent a policy interpretation
No meaningful progress possible	Escalate with attempted actions and evidence
Multiple customer/account matches	Ask for additional identifiers; do not guess based on name or heuristics
Complex but clearly within policy and tools	Complexity alone is not an automatic escalation trigger

Sentiment and self-reported confidence are poor proxies for case complexity. Use explicit criteria and examples. Distinguish a frustrated customer with a simple resolvable issue from a policy exception requiring authority.

5.3 Error propagation in multi-agent systems

A subagent should report enough information for the coordinator to choose a recovery strategy.

```
{
 "status": "partial_failure",
 "failure_type": "timeout",
```

```
  "attempted_query": "latest regulatory filing for issuer X",
  "attempts": 2,
  "partial_results": [{"claim": "...", "source_url": "..."}],
  "alternatives": ["try issuer website", "narrow date range"],
  "coverage_gaps": ["latest filing not verified"]
}
```

- Do not convert a timeout into an empty successful result.
- Do not terminate the entire workflow because one independent subagent failed.
- Recover locally from transient errors; propagate unresolved failures with partial results.
- Annotate the final synthesis with coverage gaps and unsupported areas.
- A valid empty result is different from a failed attempt to access the source.

5.4 Large codebase exploration and recovery

- Use subagents to investigate bounded questions such as 'trace refund flow dependencies' or 'find all test files'.
- Keep the main agent focused on architecture and coordination rather than dumping every file into its context.
- Persist findings in scratchpad files so later phases can reuse them.
- Summarize each phase before starting the next; inject that summary into new subagent prompts.
- For crash recovery, have agents export structured state to known locations and let the coordinator load a manifest.
- Use /compact when a long session accumulates verbose discovery output, but verify that critical facts survive compaction.

5.5 Human review and confidence calibration

A 97% aggregate accuracy figure can hide a weak document type or field. Measure accuracy by segment before automating high-confidence decisions.

- Track performance by document type, field, source quality and relevant operating segment.
- Use field-level confidence scores, then calibrate thresholds against a labeled validation set.
- Route low-confidence, ambiguous or contradictory cases to human review.
- Use stratified random sampling of high-confidence extractions to estimate residual error and detect new failure patterns.
- Do not remove human review solely because aggregate accuracy appears high.

Review segment	Example metric
Document type	Invoice vs contract vs purchase order accuracy
Field	Invoice number vs tax ID vs total accuracy
Confidence band	Observed precision for 0.90–0.95 model confidence
Error pattern	Missing currency, date ambiguity, line-item arithmetic mismatch

5.6 Provenance and uncertainty in synthesis

Summarization can destroy the link between a claim and its source. Preserve claim-source mappings through every agent handoff.

```
{
  "claim": "Revenue increased by 8%",
```

```
 "source": {
  "url": "https://example.org/report",
  "document": "Annual report",
  "page": 42,
  "published_at": "2026-03-15",
  "excerpt": "Revenue increased by..."
 },
 "data_period": "FY2025",
 "methodology_note": "Constant currency"
}
```

- Require publication or collection dates; different time periods may explain apparently conflicting values.
- When credible sources disagree, preserve both values and their source context. Do not arbitrarily select one.
- Separate established findings from contested findings.
- Preserve methodology and original source characterization.
- Report coverage gaps explicitly.
- Use the appropriate output format: financial data in tables, news in prose, technical findings in structured lists.

Integrated scenario playbooks

Scenario A — Customer support resolution agent

Architecture: coordinator/agent → MCP customer and order tools → policy gates/hooks → refund or escalation tool → structured handoff.

- Verify identity before account-specific operations.
- Gate order lookup/refund on verified customer ID where required.
- Differentiate customer lookup from order lookup in tool descriptions.
- Classify errors and retry only transient failures.
- Escalate explicit human requests, policy exceptions and cases where meaningful progress is impossible.
- Preserve order IDs, amounts, dates, verification state and attempted actions in a case-facts block.

Scenario B — Code generation with Claude Code

- Put shared standards in project CLAUDE.md; keep personal preferences in user-level config.
- Use path-specific rules for API, tests, Terraform and other file-specific conventions.
- Use plan mode for architectural or multi-file work; direct execution for simple isolated edits.
- Use project slash commands for team workflows and skills for on-demand procedures.
- Use Explore subagent to isolate verbose discovery.
- Resume only when prior context remains valid; otherwise start fresh with a structured summary.

Scenario C — Multi-agent research pipeline

- Coordinator defines scope before spawning specialists.
- Search and document analysis can run in parallel if independent.
- Pass complete findings and metadata explicitly to synthesis.
- Use structured claim-source records and dates.
- Return partial results plus error context on subagent failure.
- Run a coverage check; delegate targeted follow-up research for gaps.
- Preserve conflicting statistics rather than silently choosing one.

Scenario D — Developer productivity agent

- Start with Glob/Grep to map the codebase; Read only relevant files.
- Use built-in tools for local repository work and MCP for external systems or team-specific capabilities.
- Restrict each specialist to relevant tools.
- Persist architecture maps and flow traces in scratchpad files.
- Avoid putting all repository content into one context window.

Scenario E — Claude Code in CI

- Run non-interactively with -p / --print.
- Use JSON output and a schema for machine-readable findings.
- Provide project review and test conventions through CLAUDE.md.
- Use a separate review context from code generation.
- Keep synchronous review for blocking pre-merge gates; use batches for latency-tolerant audits.
- Deduplicate prior findings on subsequent commits.

Scenario F — Structured extraction

- Use tool_use with a JSON schema for schema-constrained output.
- Make absent fields nullable; use unclear/other patterns for ambiguity and extensibility.
- Validate semantic rules separately from JSON syntax.
- Retry with exact errors only when correction is possible.
- Use batch processing for non-blocking volumes; correlate with custom_id.
- Calibrate field-level confidence and route ambiguous cases to human review.

Exam decision framework: how to select the best answer

If the scenario says…	Think…	Usually choose…
Must always happen before another operation	Deterministic prerequisite	Programmatic gate or hook
Tools are being confused	Tool-selection ambiguity	Improve descriptions and boundaries first
Specialist uses irrelevant tools	Over-provisioning	Scope tools by agent role
Subagent cannot see prior findings	Context isolation	Pass context explicitly in task prompt
Large broad topic is incompletely covered	Bad decomposition	Fix coordinator scope map
Timeout but some findings exist	Partial failure	Structured error + partial results + alternatives
Need machine-parseable extraction	Structured output	tool_use + JSON Schema + semantic validation
Information absent from source	Evidence gap	Null/unclear or human review; do not retry endlessly
Blocking CI check	Latency requirement	Synchronous call
Nightly report	Latency tolerant	Message Batches API
Many files and architecture choices	High complexity	Plan mode, then execution
Tests scattered across folders	Cross-directory convention	Path-scoped .claude/rules glob
Long context with exact amounts/dates	Fact preservation	Persistent structured case facts
Conflicting credible sources	Provenance/uncertainty	Preserve both with dates and attribution

Rapid revision: 30 facts to know cold

1. Agentic loop: continue on stop_reason=tool_use; terminate on end_turn.
2. Append tool results to the conversation and call Claude again.
3. Do not infer completion from assistant text.
4. Coordinator owns decomposition, routing, recovery and synthesis.
5. Subagents do not inherit coordinator context automatically.
6. Pass findings and source metadata explicitly to each subagent.
7. Coordinator needs Task in allowedTools to spawn subagents.
8. Parallelize independent Task calls in one response.
9. Use hooks/programmatic gates for deterministic enforcement.
10. PostToolUse can normalize tool results before the model sees them.
11. Tool descriptions are a primary mechanism for tool selection.
12. Descriptions need purpose, inputs, examples, boundaries and outputs.
13. Structured MCP errors distinguish transient, validation, business and permission failures.
14. Valid empty results are not tool failures.
15. Too many tools reduce selection reliability; scope by role.
16. tool_choice auto permits text; any forces a tool; named selection forces a specific tool.
17. MCP tools perform operations; MCP resources expose content/catalogs.
18. Project MCP config: .mcp.json; user config: ~/.claude.json.
19. Use environment variables for credentials; do not commit secrets.
20. Grep searches content; Glob searches paths.
21. Read loads files; Edit changes unique anchored text; Write creates/replaces files; Bash runs commands.
22. Project CLAUDE.md is shared; user-level CLAUDE.md is personal.
23. Project commands live in .claude/commands/; skills in .claude/skills/.
24. context: fork isolates a skill; allowed-tools restricts access; argument-hint describes parameters.
25. Path-scoped rules use glob patterns in .claude/rules/.
26. Plan mode for complex multi-file/architectural work; direct execution for simple bounded work.
27. CI uses claude -p; structured findings use JSON output and JSON schema.
28. Few-shot examples clarify ambiguous judgment and desired output format.
29. Strict JSON schema prevents syntax errors, not semantic errors.
30. Batch API: 50% cost savings, up to 24 hours, no latency SLA, no multi-turn tool calling.

Practice exam — 25 scenario-based questions

Choose the number of responses requested. These are original practice questions based on the supplied blueprint, not actual exam questions.

1. A refund agent sometimes calls process_refund before identity verification. The system prompt already says verification is mandatory. What is the strongest fix? (Select one)

- A. Add more examples to the prompt
- B. Add a programmatic gate that blocks refund until verified identity is present
- C. Increase the model temperature
- D. Add a sentiment classifier

2. Claude requests a tool and the runtime immediately returns a final answer without executing it. What is broken? (Select one)

- A. The system prompt
- B. The loop fails to handle stop_reason=tool_use
- C. The tool schema is too strict
- D. The session name

3. A broad research topic only returns findings about two of six relevant sectors. All agents completed their assigned work. Most likely cause? (Select one)

- A. The coordinator decomposed the topic too narrowly
- B. The synthesis agent needs a larger model
- C. The search tool needs retries
- D. The output format is too short

4. A synthesis subagent says it cannot use findings found earlier by the coordinator. What should change? (Select one)

- A. Increase max_tokens
- B. Explicitly pass findings and metadata in the subagent prompt
- C. Ask it to remember
- D. Run it twice

5. Two MCP tools are repeatedly confused because their descriptions are nearly identical. Best first step? (Select one)

- A. Build a routing classifier
- B. Add more tools
- C. Rewrite descriptions with inputs, examples and boundaries
- D. Force one tool for every request

6. A search times out after returning partial results. What should the subagent return? (Select one)

- A. Empty success
- B. Generic unavailable
- C. Structured failure type, attempted query, partial results and alternatives
- D. Terminate all agents

7. Which pattern is appropriate for a synthesis agent that needs occasional simple fact checks? (Select one)

- A. Give it every search tool
- B. Give it a scoped verify_fact tool and route complex research through coordinator
- C. Remove all tools
- D. Let it fabricate likely values

8. A team-wide /review command must be available after cloning the repo. Where should it live? (Select one)

- A. ~/.claude/commands/
- B. .claude/commands/ in the project
- C. Root CLAUDE.md
- D. .mcp.json

9. Test files are scattered throughout the repository, but all need the same rule. Best approach? (Select one)

- A. Root CLAUDE.md prose only
- B. One CLAUDE.md per directory
- C. .claude/rules/ with a test-file glob
- D. Personal command

10. A 40-project .NET migration has architectural choices and cross-project dependencies. Best workflow? (Select one)

- A. Direct execution
- B. Plan mode and codebase exploration first
- C. Ask Claude to edit every file at once
- D. Avoid reading dependencies

11. A CI job hangs because Claude waits for input. Which option is identified in the guide? (Select one)

- A. --batch
- B. CLAUDE_HEADLESS=true
- C. -p / --print
- D. --silent

12. Which workflow is the best fit for Message Batches API? (Select one)

- A. Blocking pre-merge check
- B. Live refund approval
- C. Overnight technical debt report
- D. Interactive customer chat

13. A schema-valid extraction has line items totaling 950 while stated_total is 1,000. What is required? (Select one)

- A. Nothing; schema passed
- B. Semantic validation
- C. A larger context window
- D. Disable strict schema

14. A required field may be absent from the source. How should it be modeled? (Select one)

- A. Required non-null string
- B. Nullable/optional field with explicit absent-value handling
- C. Invent a likely value
- D. Retry indefinitely

15. A validation retry cannot find a tax ID because it is not in the supplied document. Best response? (Select one)

- A. Retry 20 times
- B. Return null/unclear or request the missing source
- C. Guess from company name
- D. Remove source validation

16. A model generates code and then reviews it in the same session. What is the concern? (Select one)

- A. It has too little context

- B. It retains generation reasoning and may be biased toward its prior decisions
- C. It cannot read code
- D. JSON output is invalid

17. A long case contains exact refund amounts and order IDs. How should they survive summarization? (Select one)

- A. Narrative summary only
- B. Persistent structured case-facts block
- C. Remove old messages
- D. Rely on model memory

18. Customer explicitly asks for a human agent. What should the agent do? (Select one)

- A. Investigate first
- B. Honor the request immediately
- C. Ask for sentiment score
- D. Attempt refund

19. A valid customer query returns no matching order. What is this? (Select one)

- A. Transient error
- B. Permission error
- C. Successful empty result
- D. Validation error

20. Two credible sources report different values for the same metric, measured in different periods. What should synthesis do? (Select one)

- A. Pick the newest-looking number
- B. Average them
- C. Preserve both with source and date context
- D. Drop both

21. Which tool finds all files ending in .cs? (Select one)

- A. Grep
- B. Glob
- C. Edit
- D. Bash only

22. Which tool finds every occurrence of ProcessRefund in file contents? (Select one)

- A. Glob
- B. Grep
- C. Write
- D. Read only

23. Which tool_choice requires a tool call but allows the model to choose among available tools? (Select one)

- A. auto
- B. any
- C. end_turn
- D. tool_use

24. Which is the best way to measure residual error in high-confidence extractions? (Select one)

- A. Trust model confidence
- B. Stratified random sampling of high-confidence outputs
- C. Review only low-confidence outputs
- D. Use aggregate accuracy only

25. A convention must apply to every Terraform file, wherever located. What should be used? (Select one)

- A. A directory-only CLAUDE.md
- B. Path-scoped rule with terraform glob
- C. User-level command
- D. A tool resource

Practice exam answer key and reasoning

1. Answer: B. A prompt is probabilistic. A financial prerequisite needs deterministic enforcement.

2. Answer: B. Execute the requested tool, append its result and continue the loop.

3. Answer: A. The coordinator's scope decomposition determines coverage.

4. Answer: B. Subagents do not automatically inherit parent context.

5. Answer: C. Fix the source of ambiguity before adding infrastructure.

6. Answer: C. Structured context lets the coordinator recover intelligently.

7. Answer: B. Scoped cross-role access supports common simple cases without over-provisioning.

8. Answer: B. Project-scoped commands are version-controlled.

9. Answer: C. Glob path-scoped rules cover matching files regardless of directory.

10. Answer: B. Complex multi-file architectural work benefits from exploration and planning.

11. Answer: C. The print flag runs non-interactively.

12. Answer: C. Batch processing is latency tolerant and has no guaranteed latency SLA.

13. Answer: B. Schema validity does not guarantee semantic consistency.

14. Answer: B. Nullable fields reduce pressure to fabricate missing facts.

15. Answer: B. Retries cannot recover absent information.

16. Answer: B. Use an independent review instance.

17. Answer: B. Preserve exact transactional facts separately.

18. Answer: B. The guide says explicit human requests should be honored immediately.

19. Answer: C. An empty result is not an access failure.

20. Answer: C. Preserve provenance and temporal context; do not silently reconcile.

21. Answer: B. Glob matches file paths.

22. Answer: B. Grep searches content.

23. Answer: B. any forces a tool call without forcing a particular named tool.

24. Answer: B. Sampling high-confidence outputs detects hidden errors and novel patterns.

25. Answer: B. Path-scoped glob rules apply across directories.

Hands-on preparation labs

Lab 1 — Customer-support agent

- Define get_customer, lookup_order, process_refund and escalate_to_human tools.
- Write non-overlapping descriptions with explicit identifiers, outputs and boundaries.
- Implement the agent loop using stop_reason.
- Add a verified-customer prerequisite and refund-threshold hook.
- Return structured transient, validation, business and permission errors.
- Test explicit human request, multiple account matches, policy exception and multi-issue requests.

Lab 2 — Claude Code team configuration

- Create project CLAUDE.md with build, test, architecture and review conventions.
- Add path-scoped rules for API, tests and Terraform files.
- Create a project command such as /review and an isolated skill using context: fork.
- Configure a project MCP server using environment-variable credentials.
- Compare plan mode and direct execution on a single-file fix and a multi-project migration.

Lab 3 — Structured extraction

- Create a JSON schema for invoices or contracts with nullable fields and extensible enums.
- Extract a small sample with varied layouts.
- Run schema, semantic and source-evidence validation separately.
- Retry only format/structure failures; route missing evidence to null or human review.
- Track accuracy by document type and field; sample high-confidence results.

Lab 4 — Multi-agent research

- Build a coordinator, search subagent, document-analysis subagent and synthesis subagent.
- Pass complete findings and metadata explicitly.
- Run independent research tasks in parallel.
- Simulate a timeout and preserve partial results and coverage gaps.
- Introduce conflicting source statistics and ensure both remain attributed in the final report.

Suggested 14-day study plan

Day	Study focus	Deliverable
1	Exam blueprint and agentic loop	Draw loop; explain stop_reason and tool result cycle
2	Coordinator/subagent patterns	Design a multi-agent architecture and handoff contract
3	Hooks, gates, decomposition and sessions	Implement a deterministic prerequisite and recovery flow
4	Tool descriptions and MCP error design	Write 4 tool contracts and structured error taxonomy
5	MCP servers/resources and built-in tools	Configure a project server; map tool vs resource
6	CLAUDE.md hierarchy and path rules	Create project instructions and glob-scoped rules
7	Commands, skills, plan mode and iteration	Build one command and one isolated skill
8	CI/CD and independent review	Design a CI review job with schema output
9	Explicit prompts and few-shot examples	Write review criteria plus positive/negative examples
10	JSON Schema and validation	Build extraction schema and semantic checks
11	Batch processing and multi-pass review	Choose sync vs batch for five workloads
12	Context, escalation and error propagation	Design case facts, scratchpad and escalation contract

13	Confidence and provenance	Build sampling plan and claim-source mapping
14	Mock exam and weak-area review	Complete 25 questions; revisit every incorrect answer

For a shorter preparation window, combine adjacent days but retain the hands-on labs and scenario practice. Weight study time approximately in line with the exam domain percentages.

Supplementary expansion — community resource synthesis

Purpose. This section expands the original study guide using the resources supplied by the candidate. The official July 2026 exam guide remains authoritative for exam scope and terminology. Community resources are unofficial study aids, not evidence of live exam questions or scoring.

1. Resource review and source quality

Resource	Contribution identified	Use in preparation
ClaudeCertificationGuide /learn	Five-domain curriculum, 30 task statements, practice navigation, diagnostic, glossary	Curriculum map and domain drills
Paolo Amato practice exam	60 original scenario questions, detailed rationales, distractor analysis, quick review and version notes	Full-length scenario practice; not psychometrically calibrated
Szymon Paluch practice exam	60 original scenario questions, study/exam modes and domain breakdown	Timed practice; single-answer only, unlike the official guide's mixed item formats
Neerajkr7 repository	200 scenario questions, filters, explanations, bookmarks and missed-question log	High-volume drilling by domain
Hamza Farooq repository	Five domain cheat sheets, 64-question mock, sample questions	Detailed domain-level revision and trap analysis
Ahmed Hasan repository	Concept map, trap guide, anti-pattern catalogue and product FAQ	Last-stage revision; verify product behavior against current docs
Avi Develops repository	Scenario explanations covering semantic extraction, orchestration, context, batch and recovery	Architectural reasoning examples
Sunbhanu repository	URL could not be retrieved for inspection in this review	Not used as a factual basis

Source hierarchy Use the official exam guide for what is tested. Use official current Anthropic/MCP documentation for implementation details. Use community mocks to practise judgment, not to infer actual exam content or scaled-score conversion.

2. Cross-domain rules that repeatedly decide scenario questions

Rule	How to apply it	Example
Fix the root cause at the cheapest effective layer	Do not add infrastructure before correcting the existing layer	Tool misrouting → improve descriptions before building a classifier
Deterministic controls for high-impact invariants	Prompts guide; programmatic gates enforce	Identity verification before refund → gate the refund tool
Trace failures upstream	A downstream agent may have executed its task correctly from a bad brief	Missing research sectors → fix coordinator decomposition
Structure high-value state	Facts that must survive summarization need a structured representation	Case-facts block with IDs, amounts, dates and statuses
Least privilege	Give each agent only the tools it needs	Synthesis gets constrained verify_fact, not unrestricted search
Preserve partial value	One subagent failure should not erase successful independent work	Return partial results plus coverage gaps
Preserve provenance	Attribution must travel with each claim	Claim + URL + document + excerpt + date

Match mechanism to failure	Different failure classes need different fixes	Few-shot for inconsistent judgment; schema for shape; validation for semantics

3. Expanded anti-pattern catalogue

Tempting but weak approach	Why it fails	Preferred approach
Parse assistant prose to end an agent loop	Natural language is not a reliable control signal	Use stop_reason and the tool-result cycle
Use arbitrary iteration cap as primary completion logic	May stop early or fail to detect completion	Use stop_reason; retain bounded safety guards for runaway/repeated calls
Micromanage subagents with rigid procedural steps	Blocks adaptation to evidence	Specify goal, scope, output contract and quality criteria
Narrowly decompose a broad topic	Creates systematic coverage gaps	Build a scope/coverage map before delegation
Let subagents coordinate directly	Weakens observability and error control	Route coordination through coordinator
Use prompt-only ordering for financial actions	Prompt compliance is probabilistic	Programmatic gate or tool-call interception
Parallelize an unknown bug investigation blindly	Wastes work on irrelevant areas	Map codebase, then adapt subtasks
Resume with stale tool results	Agent may act on obsolete code/data	Start fresh with structured summary if findings are stale
Keep generic overlapping tools	The model cannot reliably distinguish purposes	Split or differentiate tools with tight contracts
Use keyword-based routing instructions	May conflict with tool descriptions	Clarify descriptions and remove conflicting prompt associations
Return generic operation failed	No recovery decision can be made	Return error category, retryability, attempted action, partial results and alternatives
Treat empty result as tool failure	Causes false retries and escalation	Distinguish successful no-match from access failure
Give every agent every tool	Increases selection complexity and misuse	Scope tools by role
Give synthesis unrestricted web search	Violates separation of concerns and expands attack surface	Give a constrained verify_fact tool for simple lookups
Commit real MCP credentials	Secrets leak through version control	Commit environment-variable references, not values
Search only original function name through wrappers	Misses aliases/re-exports	Inspect exports and wrappers, then search exported names
Put team standards in user-level CLAUDE.md	Teammates do not receive them	Commit project-level instructions
Use one giant CLAUDE.md	Token bloat and poor maintainability	Use imports and focused rules
Use a skill for universal always-on standards	Skills are on-demand workflows	Use CLAUDE.md for universal instructions
Use plan mode for a tiny isolated fix	Adds unnecessary overhead	Direct execution for simple bounded changes
Ask code generator to review itself in same session	Generation context creates review bias	Use an independent review instance

Say only 'be conservative'	No decision boundary is defined	Specify what to flag, report and skip
Require fields absent from source	Encourages fabrication	Use nullable/optional fields and unclear states
Retry when information is absent	Retry cannot create missing evidence	Return null/unclear or request missing source
Use batch for blocking pre-merge checks	Up to 24-hour processing with no latency SLA	Use synchronous request
Review a large change in one huge pass	Attention dilution and inconsistent depth	Per-file pass plus cross-file integration pass
Drop old turns with a sliding window in multi-issue support	Resolved topics may be reopened	Summarize resolved threads; retain active thread and case facts
Append every tool field	Context bloat buries relevant facts	Trim tool output before appending
Guess identity from name or confidence	Can select wrong account	Ask for a unique identifier
Escalate only because customer is angry	Sentiment is not case complexity	Acknowledge and offer resolution; escalate if human request is reiterated
Suppress subagent errors as empty success	Coordinator cannot distinguish failure from no matches	Propagate structured error and partial results
Terminate whole workflow on one subagent failure	Throws away successful work	Recover and annotate coverage gaps
Clear context periodically without preserving findings	Destroys useful discoveries	Use scratchpads, phase summaries and manifests
Trust aggregate accuracy alone	Can hide weak document types/fields	Segment evaluation and sample high-confidence outputs
Summarize without claim-source mapping	Attribution disappears	Carry structured provenance through synthesis
Choose one conflicting statistic silently	Hides uncertainty	Preserve both with source, date and method

4. High-value nuances missing or under-emphasized in the first edition

4.1 Primary loop control versus defensive loop guards

stop_reason remains the primary control-flow signal: continue for tool_use and finish for end_turn. A separate defensive guard can detect repeated identical tool calls, no progress, excessive cost or a wall-clock budget breach and trigger a controlled escalation. Do not confuse a safety guard with the completion signal. The official guide rejects arbitrary iteration caps as the primary stopping mechanism; production systems still need protection from runaway loops.

4.2 Forced tool choice for a mandatory first action

If a specific tool must run first, force that named tool. tool_choice='any' only guarantees some tool call, not the required tool. After the mandatory first operation, return to auto selection for subsequent decisions.

4.3 Retryability is conditional

- Transient: retry the same input within a bounded retry budget, commonly with backoff.
- Validation: retry only after correcting the invalid input.
- Business: do not retry unchanged; explain the rule or route to an exception workflow.
- Permission: do not blindly retry with unchanged authority; request appropriate authorization or escalate.
- Valid empty result: do not retry as though the tool failed.

4.4 Narrow tool interfaces reduce blast radius

Prefer constrained domain operations over powerful generic tools. Examples: query_orders(filters) instead of unrestricted run_sql(query); send_customer_notification(template_id, ...) instead of unrestricted send_email(...); load_document(document_url) with domain/content-type validation instead of generic fetch_url(url). This applies least privilege at the tool-contract level.

4.5 Multi-issue context strategy

Do not use a sliding window that simply discards the oldest turns when a customer may return to a resolved issue. Maintain structured facts for open and recently resolved issues. Summarize resolved discussion, retain the active issue in detail, and preserve exact identifiers, amounts, dates and commitments in a facts block.

4.6 Confidence is calibrated, not truth

Raw model confidence should not authorize an irreversible action or decide identity. For review routing, calibrate field-level confidence against a labeled validation set by document type and field. Continue stratified sampling of high-confidence cases after deployment to detect drift and novel error patterns.

4.7 Batch SLA planning

The exam guide states up to 24 hours of processing with no guaranteed latency SLA. Work backward from the end-to-end SLA, leaving time for polling, failed-item resubmission and human review. Correlate results with custom_id and resubmit only failed records.

4.8 Semantic extraction must preserve stated source values

For invoices, extract stated_total and calculate calculated_total from line items. Do not silently alter line items to force a match. Mark conflict_detected and route discrepancies for review. This separates source extraction from arithmetic validation.

5. Exam-guide terminology versus current product documentation

Keep two views separate Use the July 2026 Exam Guide terminology for exam questions. Use current Anthropic documentation for production implementation. When names or request shapes differ, record the version boundary rather than silently rewriting the exam baseline.

Topic	Exam-guide baseline	Current documentation note
Subagent invocation	Task tool and Task in allowedTools	SDK interfaces evolve; verify current Agent SDK subagent API and tool names before coding
Structured output	tool_use with JSON Schema and tool_choice	Current API distinguishes JSON output via output_config.format from strict tool use via strict:true; they address response format versus tool input validation
Schema correctness	Schema-constrained output prevents syntax/shape failures	Semantic correctness still requires application validation
Claude Code CLI	-p/--print, --output-format json, --json-schema	CLI flags are version-sensitive; verify installed CLI help and current docs
MCP configuration	.mcp.json project scope and ~/.claude.json user scope	Treat as exam-guide baseline; verify current config schema before deployment

Current Anthropic Structured Outputs documentation states that JSON outputs use output_config.format with type json_schema, while strict tool use validates tool names and inputs. They can be used independently or

together. The current page also lists JSON Schema limitations; a schema accepted by a generic validator may not be accepted by the API. Reference: https://platform.claude.com/docs/en/build-with-claude/structured-outputs.

6. Scenario decision trees

Symptom	Diagnostic question	Preferred response
Repeated tool calls	Is the runtime returning tool results and is progress occurring?	Correct the tool-result loop; add repeated-call/no-progress guard and escalate if stalled
Mandatory first tool skipped	Must one specific tool run before any other action?	Force the named tool; do not rely only on prompt text
Wrong tool selected	Are descriptions overlapping or inputs ambiguous?	Clarify names, inputs, examples, outputs and boundaries
Too many tools	Are tools outside the role exposed?	Reduce tool surface and create constrained specialist tools
Research misses areas	Did coordinator cover the full topic?	Fix decomposition before changing downstream agents
Subagent lacks prior facts	Was context explicitly passed?	Include findings and metadata in task prompt
No matching record	Did the tool access the source successfully?	Empty success if access succeeded; structured error if access failed
Long conversation loses exact facts	Are facts only present in prose?	Persistent structured case-facts block
Extraction field missing	Was it present in the source?	Absent → null/unclear; present but malformed → retry with precise feedback
Totals do not reconcile	Syntax or semantic failure?	Calculate independently; preserve stated/calculated totals; flag mismatch
Review has false positives	Are categories defined with concrete include/exclude criteria?	Categorical criteria; disable noisy category temporarily if needed
Self-review misses defects	Is reviewer independent of generator context?	Second independent instance and focused passes
Batch job misses SLA	Blocking or latency tolerant?	Synchronous for blocking; batch for non-blocking with failure buffer
Subagent fails midway	Can partial results be preserved?	Structured error + partial results + alternatives; coordinator continues safely
Conflicting research values	Same time period, method and population?	Preserve both claims with provenance and temporal context

7. Advanced practice questions — additional scenario traps

26. An agent repeats lookup_order with the same identifier after receiving an empty successful result. Best correction? (Select one)

- A. Treat every empty result as transient and retry
- B. Preserve empty-success semantics, detect repeated/no-progress behavior as a safety condition, then ask for a different identifier or escalate
- C. Increase max_tokens
- D. Change temperature

27. A workflow must call extract_intent before routing. A prompt instruction is skipped in 15% of runs. Strongest first-turn control? (Select one)

- A. tool_choice auto
- B. tool_choice any
- C. Force the named extract_intent tool
- D. Add more system prompt text

28. A generic tool handles refunds, store credit, exchanges and partial refunds through a mode field. Output quality is inconsistent. Best design? (Select one)

- A. Longer description only
- B. Split into purpose-specific tools with distinct contracts
- C. Increase context window
- D. Add a second model to select the mode

29. Project MCP configuration must be shared, but each developer has a different token. Correct approach? (Select one)

- A. Commit each token
- B. Move all config to user scope
- C. Commit project config with environment-variable references
- D. Remove authentication

30. An analysis skill must be read-only. What constrains its tool access? (Select one)

- A. context: fork
- B. allowed-tools limited to read/search tools
- C. argument-hint
- D. User-level CLAUDE.md

31. Customer is upset about a delayed order but has not requested a human. The issue is within policy and remedy is available. What should the agent do? (Select one)

- A. Escalate based on sentiment
- B. Acknowledge frustration and offer the permitted resolution
- C. Ask customer to repeat the complaint
- D. Route to supervisor because case is emotional

32. Source document contains no renewal date. Safest extraction output? (Select one)

- A. Infer from contract start date
- B. Return null/unclear and preserve missing-evidence state
- C. Retry until date appears
- D. Use common renewal period

33. Accuracy is 97% on English digital invoices, but production includes scans and German invoices. First evaluation improvement? (Select one)

- A. Add more English digital samples
- B. Segment evaluation using representative formats and languages
- C. Remove low-confidence cases
- D. Treat model confidence as accuracy

34. Search subagent fails while document analysis succeeds. Coordinator response? (Select one)

- A. Terminate report
- B. Treat search as empty success
- C. Use successful findings, consider fallback/retry and annotate coverage gap
- D. Ask synthesis to invent citations

35. A 20-file review misses cross-file data flow defects. Best redesign? (Select one)

- A. One much longer prompt
- B. Per-file analysis plus a separate integration pass
- C. Three full reviews and majority vote
- D. Disable cross-file checks

36. CI review repeats old comments after every new commit. What should be added? (Select one)

- A. Include prior findings and ask for only new or unaddressed issues
- B. Increase output tokens
- C. Change temperature
- D. Delete test suite

37. Tool returns 40 fields, but next step needs status, amount and order date. What should happen? (Select one)

- A. Append entire response
- B. Trim before adding to context
- C. Ask model to ignore fields
- D. Summarize all fields

38. Two source values differ. What metadata is essential before calling them contradictory? (Select one)

- A. Agent confidence
- B. Publication/data collection date and methodology/population
- C. Search query length
- D. Number of subagents

39. A long code exploration crashes. What supports robust recovery? (Select one)

- A. Chat history only
- B. Structured state exports and a manifest loaded by coordinator
- C. Re-run all commands blindly
- D. Increase temperature

40. A noisy review category damages trust in all findings. Best immediate action? (Select one)

- A. Keep reporting but lower severity
- B. Temporarily disable category while refining criteria
- C. Ask model to be careful
- D. Add more categories

Advanced practice answer key

26. Answer: B. An empty successful lookup is not an access failure. Repeated no-progress calls need a bounded recovery path.

27. Answer: C. Forced named selection guarantees that tool; any only guarantees some tool.

28. Answer: B. Different operations and side effects warrant purpose-specific contracts.

29. Answer: C. Share integration configuration, not secret values.

30. Answer: B. allowed-tools is the permission surface; fork isolates context.

31. Answer: B. Frustration alone is not an escalation trigger.

32. Answer: B. Missing source information must not be fabricated.

33. Answer: B. Aggregate accuracy masks segment weakness; evaluation must mirror production.

34. Answer: C. Graceful degradation preserves partial value and exposes uncertainty.

35. Answer: B. Focused local passes preserve depth; integration pass catches cross-file behavior.

36. Answer: A. Prior findings enable deduplication across review runs.

37. Answer: B. Filter at the tool/runtime boundary before context accumulation.

38. Answer: B. Temporal and methodological context may explain differences.

39. Answer: B. A manifest is a durable checkpoint independent of chat history.

40. Answer: B. Suppress the noisy category until it meets acceptable precision.

8. Recommended practice-resource workflow

Stage	Resource/activity	Purpose
Baseline	ClaudeCertificationGuide diagnostic or one 60-question mock	Find weak domains before studying
Concept learning	This guide plus official exam guide	Understand mechanisms and task objectives
Trap revision	Ahmed Hasan anti-pattern/trap catalogue	Learn why plausible distractors fail
Domain drilling	Neerajkr7 200-question bank and Hamza cheat sheets	Repeat weak concepts
Full simulation	Paolo Amato or Szymon Paluch 60-question mock	Practise pacing and scenario analysis
Review	Every wrong or guessed answer	Explain root cause and why each distractor is weaker
Hands-on	Build agent loop, MCP tools, Claude Code config and extraction flow	Convert terminology into operational understanding

Mock score warning Community mocks use their own scoring formulas and may be single-answer only. A mock score is a diagnostic signal, not a conversion to Anthropic's scaled score or a guarantee of passing.

9. Final readiness checklist

- I can explain the agentic loop and stop_reason-driven continuation.
- I can design a coordinator that decomposes broad work, passes context and checks coverage.
- I know when hooks, gates or forced tool choice are stronger than prompts.
- I can distinguish tool-description defects, tool overload and poor tool boundaries.
- I can classify transient, validation, business and permission errors and distinguish empty success.
- I can explain MCP tools versus resources and project versus user configuration.
- I can choose Glob, Grep, Read, Edit, Write and Bash correctly.
- I can explain CLAUDE.md hierarchy, rules, commands, skills, frontmatter and path scoping.
- I can choose plan mode versus direct execution based on scope and risk.
- I can configure non-interactive CI use and structured output with an independent review context.
- I can write categorical review criteria and targeted few-shot examples.
- I can design nullable schemas, validate semantics and avoid retries when evidence is absent.
- I can choose synchronous versus batch processing from latency/SLA constraints.
- I can preserve case facts, partial results, manifests and provenance across context boundaries.
- I can explain escalation triggers, confidence calibration and stratified sampling.
- I can handle conflicting source claims without silently selecting one.
- I have practised both single-answer and multiple-response questions.